

To boldly suggest an overall plan for C++26

First things first

C++ ships on time.

C++11 shipped late, C++14 shipped on time, C++17 shipped on time, C++20 shipped on time, C++23 will ship on time, C++26 will ship on time.

This plan doesn't change any of that. What is ready to ship can ship; what is not, will need to take the next train or a separate ship vehicle.

If a planned item cannot make it on time, we will postpone it; if it's in the Working Draft and deemed not ready, we will rip it out and ship the IS on time.

No, you don't need to get plenary approval to get a paper in the second or third priority bucket discussed

Just because execution, ranges, reflection and pattern matching are high-priority items doesn't mean and will not mean that they will exhaust all time available. If we are realistic, we will admit to ourselves that while they are high-priority items, their priority doesn't prevent other material from being discussed; we will look at the proposals related to them, review them, and send said proposals to be revised. We are not going to just idly chat about them without doing anything else while the proposals are being revised.

Abstract

In a nutshell, the plan is thus: for C++26, let's work towards having the following things in that standard:

- Execution
- More Ranges work
- Reflection

Without a particular ship vehicle yet, we should also make progress on

- Contracts
- Pattern matching

All good? Agreed? Right, carry on. :) In case you want some elaboration, read on. I will elaborate on what C++26 Must Focus On and things that should progress during the standardization of C++26, but do not necessarily need to be in C++26 as the ship vehicle.

An attempt at a brief explanation on what this plan is for, and what it's not for

This plan is a focus plan, and is based on existing material. The purpose of this plan is not to make statements about "C++ should be able to do foo". Direction's P2000 may be, but this plan is about what we should make progress with from what we already have in some form when the C++26 cycle begins. In other words, this plan is not a wish list. The purpose of this plan has always been to be more concrete than that. That has its downsides; this plan doesn't solve the problem of how to make a wish list item materialize. But this plan was never designed to do that, either.

Or to phrase it differently: P2000 and especially its Section 5, Medium-term Aims (3-10 years) is more "someone should work on this", and that the term is indeed Medium, whereas this plan is about "we can reasonably expect that we know people who will work on these", and the term is one standard cycle.

High-level priority order, see here if you're concerned about issues and bug fixes

The priority order of handling material is thus:

1. Material that is mentioned in this plan.
2. Bug fixes, performance improvements, integration fixes for/between existing features, and issue processing.
3. Material that is not mentioned in this plan.

This means three things:

- Design groups will discuss material in that priority order.
- Design groups will, when sending material forward for wording review, designate which bucket the material belongs to.
- Wording review groups can use this priority order and said designation to prioritize their own work.

The priority order doesn't mean that bug fixes can't be considered before all on-plan material has been completely processed, or that off-plan papers can't be looked at while there are open issues. It does, however, allow wording-review groups to do issue processing even in a face-to-face meeting before looking at an off-plan paper, subject to the wording-review group's own prioritization.

Flushing the pipeline of features that we originally targeted for C++23 but couldn't get into C++23 is either at the boundary of (2) and (3), or just in (3), and that seems just fine.

C++26 must-work-on-first items

Any C++26-era WG21 subgroup meeting works on no other *NEW* material until all possible progress on these items is exhausted first

As stated already, "all possible progress" means at most a day or two of a face-to-face meeting, not all of a face-to-face meeting. Also note the emphasis on "new".

It is expected that some bugfixes and minor revisions of smaller proposals practically get higher priority after design deadlines have passed, and we're in the finishing stages. That is, if after a design deadline we have a proposal that still needs design work and design reviews, it's perfectly okay to (slightly, but not completely) shift the focus to revisions of smaller proposals that merely need small tweaks to get on a shipping train. The actual judgement on this is expected to be exercised by the leaders of the design groups.

As stated in the abstract, these items are

- Execution
- More Ranges work
- Reflection

Execution

During the C++23 cycle, we made good progress on Senders and Receivers. The lofty expectation is that we will merge most of the foundational bits ([P2300](#)) early in the C++26 cycle. There is, however, more to be done in that area, once the rudimentary first steps are in:

- more sender adaptors
- async scopes
- async streams

More Ranges work

We have a good master plan for ranges, and it's expected that more of that work will land in C++26. The plan is in [P2214](#). We expect to see a plan update for C++26.

Reflection

First, see Dr. Sutton's overview in [P2237](#).

There's a whole host of use cases and programmer needs that "same-language metaprogramming" covers. We should make an effort to ship the first language and library parts of it in C++26.

Items that should make progress, but can land later than C++26, or can land in a separate ship vehicle

Nota Bene: as is always the case with our train model, proposals that are ready in time can be adopted. Thus this plan doesn't preclude Contracts or Pattern Matching from shipping in C++26. This is just a relative priority, considering the unclear foreseeable maturity of these proposals.

Contracts

Redesigning Contracts has made some progress during the C++23 cycle, but there are various unresolved issues at the Study Group level, so it's hard to foresee at this point what the pace of that work will be. It is, however, deemed reasonable to make progress in that area an on-plan item for this plan.

Pattern matching

While we often say that C++ needs new control statements like it needs a hole in the head, pattern matching shows a lot of promise to deliver better filters/chains/selections than what we can approximate with pure-library facilities. Pattern matching improves type safety by making it easier to write type-safe code than writing the conventional C-style or traditional-C++-style alternatives. Also, it puts the visitor pattern out of business in many cases. It's not just another control structure; it interacts with the type system.

There's fair amounts of ongoing work in this area, and we should give it air time during the C++26 time frame. How to ship it, and in what form, is again not entirely clear yet.

Where's library support for Coroutines, which was in the C++23 plan?

While we can still expect to see proposals for a task type and other such specific things, the execution proposal, P2300, covers generic library support for coroutines, and knits them into the generic asynchrony framework provided by P2300.

Where's Networking, which was in the C++23 plan?

The earlier direction/approach for Networking didn't have consensus, to a fair extent due to concerns about the composability of the approach. We don't have a composable approach proposal at this point.

Should such a proposal arrive, we can reconsider its priority in this plan; but we can also ostensibly just handle it as a normal-priority material item, like we do for everything else. But all that is moot as long as we have no idea whether such a proposal will arrive.

What about linear algebra?

BLAS-based linear algebra is making good progress. It doesn't seem to need a special scheduling priority, and can be dealt with as an extension almost like any other, except that the LEWG chairpeople know to give it relatively high priority among extensions.

What about my favorite idea X, Y, foo and bar?

We need a plan on which we focus, to make sure that the important things that we need to deliver are delivered. In order to introduce new high-priority items into a plan like this, feel free to try and convince the committee that your favorite extension is

1. truly so important that it is either a must-work-on-first item or must-progress item
2. a good idea in general.

And again, if you have a proposal that is important, make sure it has a good rationale, and it will be discussed, either as an off-plan proposal (to which we will get during C++26 even if we have material with higher priorities) or as a priority 2 proposal.

As a realistic but yet vague example of this, there have been multiple requests to put safety-critical programming in C++ into this plan. However, this plan is, again, not a plan for pipe dreams. This is a focus plan. It's grounded on things that more or less already exist, and it's not an invitation to work on particular areas that don't yet have concrete proposals. P2000 may be such an invitation, but that's never been the purpose of this plan.